Hardware Verification Using Model Checking and Formal Logic: An Experience Report

Larry Lee

July 16, 2020

Abstract

In this article, I describe hardware verification using model checking and formal logic. I compare and contrast these two methodologies and show that both methodologies are fundamentally equivalent - i.e. that model checking is a special case of theorem proving. Specifically, model checking corresponds to a proof technique called Case Analysis. Additionally, I identify the practical and theoretical limitations of these methods and explain how these methodologies can be combined in a complementary fashion to mitigate these deficiencies. Finally, I close by presenting case studies in which model checking and formal logic were used successfully to verify a portion of the Kami processor and components used within SiFive's S-series processors.

Background

In this article, I review formal model checking and compare it with the use of formal logic for system verification.

Hardware verification is the practice of ensuring that a hardware system conforms to a specification. A hardware specification prescribes a set of constraints that must by satisfied by a system. A specification must define the intended behavior of the system and may constrain additional attributes of a computation process, such as its execution time and memory resources.

Every verification methodology encompasses five core activities:

- Specification
- Modeling
- Constraint Translation
- Deriviation
- and Composition.

Verification methodologies differ based on the methods that they use to perform these activities, but they all prescribe methods to perform them. As a result, we can characterize and compare verification methodologies based on their approach to these five activities. In this article, I examine the ways in which methodologies based on model checking and formal logic perform these activities, and argue that model checking and formal logic are fundamentally complementary approaches.

Every verification methodology starts with specification - the formal definition of the constraints that must be satisfied by a conformant system. These specifications must define the functional behavior of the system - its input domain and output range and the mapping between them - and constrain additional computation properties such as time, memory, area, and power consumption.

Once these constraints have been formulated, every verification methodology must formally model the system being verified. At Apple, hardware systems are modeled using System Verilog. At SiFive, I modeled systems using mathematical equations and the hardware modeling language, Kami.

Once we have modeled the system, every hardware methodology requires us to translate the constraints prescribed by the system specification into model constraints such as System Verilog Assertions, mathematical equations, or logical propositions.

Once this is done, each methodology requires us to apply a set of methods to verify that the model indeed satisfies the identified constraints. This process varies based on the methodology used. Model checking uses a variant of stimulus testing in which inputs are fed into the model and output is observed to infer facts about the behavior of the system under analysis.

All of these methods become increasingly difficult to apply as the unit under analysis grows in size and complexity. Accordingly, all of these methods require us to decompose larger systems into subunits that can be verified independently.

By decomposing larger systems into modules - subunits that interact with each other through interfaces - we are able to use the principles of abstraction, encapsulation, and decomposition to simplify our analysis. However, after we decompose a system into modules, we must then combine results about these modules to derive assurances about the system as a whole.

Illustrative Example

Throughout the rest of this article I will be using a simple hardware system to illustrate the use of model checking and of formal logic for hardware verification. In whath follows, I will refer to this system as the Divisibility Checker. The Divisibility Checker accepts a sequence of bits and outputs a single bit that indicates whether or not the number represented by the input sequence is divisible by 3.

In figure 1, I present a state transition diagram for the Divisibility Checker. As the unit receives bits on its serial bus, it updates its internal state to one of three states. These states are labeled, 0, 1, and 2. The device maintains a simple invariant in which the current state is congruent to the number received so far

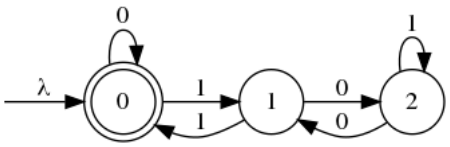


Figure 1: The state transition diagram for the Divisibility Checker.

mod 3. For example, the bit sequence [1, 0] represents the binary number 2, which is congruent to 2 mod 3. When reset, the device enters state 0. Upon receiving the first bit, it transitions to state 1, because the number received represents the number 1 which is congruent to 1 mod 3. Next the module receives a 0, so the bits represent the binary number 2, which is congruent to 2 mod 3. Accordingly the device moves to state 2. If the device is in state 0, it outputs a 1 indicating that the number received is divisible by 3. Otherwise it outputs a 0.

In Figure 2, I present Gallina code that implements this abstract finite state machine. This code sample instantiates an instance of the finite state machine record type, which is defined in my fsm library. You can find the full source code for both the fsm library and the abstract Divisibility Checker at: https://github.com/llee454/fsm.

```
Definition fsm := {|
  State   := FsmS;
  Input   := Bit;
  Output  := Bit;
  initial := SZero;
  output  := fun s => match s with SZero => One | _ => Zero end;
  transition :=
    fun s b =>
      match s with
      | SZero => if b then SZero else SOne
      | SOne  => if b then STwo  else SZero
      | STwo  => if b then SOne  else STwo
      end
|}.
```

Figure 2: A Gallina model of the Divisibility Checker.

In figure 3, I present Kami code that implements this device. In this formalization, you see that the device contains a 2 bit register capable of storing three state values and has three ports: a serial input line, a single bit output port, and a reset line. When the reset line is held low, the device reads a bit from the input port on the leading edge of each clock signal. Additionally, it outputs a signal indicating whether or not the currently received value is divisible by 3.

```
Definition FSMMod : Mod :=
  Base (MODULE {
    Register stateReg : State
      <- ConstBit ($StateMod0 : word stateSz) with
    Rule "main" :=
      Read prevState : State <- stateReg;
      Call reset : Bool <- "reset" ();
      Call input : Maybe (Bit 1) <- "getInput" ();
      LET state : State
        <- IF #reset
             then $StateMod0
             else IF #input @% "valid"
                 then transition #prevState (#input @% "data")
                 else #prevState;
      Write stateReg : State <- #state;
      Call "sendOutput" (#state == $StateMod0 : Bool);
      Retv
  }).
```

where the transition function is defined as follows:

```
  Definition transition (state : State @# ty) (b : Bit 1 @# ty) :
    State @# ty :=
    Switch state Retn State With {
      ($StateMod0 : State @# ty) ::=
        IF b == $0 then $StateMod0 else $StateMod1 : State @# ty;
      ($StateMod1 : State @# ty) ::=
        IF b == $0 then $StateMod2 else $StateMod0 : State @# ty;
      ($StateMod2 : State @# ty) ::=
        IF b == $0 then $StateMod1 else $StateMod2 : State @# ty
    }.
```

Figure 3: A Kami implementation of the Divisibility Checker.

Model Checking

Model checking is the practice of verifying that a hardware design is correct by observing the design's response to a range of inputs. Ideally, the inputs used would be exhaustive - i.e. every possible combination of input and output values would be observed. However, model checking is constrained by combinatorial explosion. The set of input/output combinations for even modestly sized functional domains are too large to test exhaustively since the size of the verification space grows exponentially with the size of the functional domain.

Specification

In model checking specifications can be either property based or model based. In property based formal verification, we identify a set of logical propositions that characterize the input and output domains. Ideally, the set of input and output propositions are specific enough to uniquely define the system's function set. However, in many applications, the output value returned by the system in response for values that fall outside of the system's input domain are undefined.

Alternately, the functional behavior of a system may be embodied by a reference implementation. At SiFive, we used the idea of a "transparently correct implementation" as part of our verification activities. Given a reference implementation, we effectively, declared that the input/output behavior of the reference implementation defined the behavior function of our implementation.

In practice, a "transparently correct implementation" is an evaluable model of the system that defines the function space of the system in a naive or obvious way whereas our target implementation may contain optimizations that render static analysis difficult.

Constraint Translation

Given a System Verilog model, property based specifications may be encoded as System Verilog Assertions. In Kami, logical propositions can be expressed directly as assertions within Gallina.

When using model based specification, constraint translation refers to the act of defining the reference model itself, and selecting input/output values for stimulus testing.

While at SiFive, I developed several such models. Significantly, Kami Processor was created to provide a canonical reference model for the RISC-V community.

Derivation

Model checking uses direct verification by testing the system on specific inputs and observing the output. Each test however only confirms that the system will generate the correct output for the specific set of inputs selected for the test.

In practice, the test set constitutes a subset of the system's input domain. Accordingly, model checking requires testers to use logical induction to extrapolate results.

The primary principle of input selection is to identify edge cases, points within the input domain at or near behavioral boundaries - points within the input domain near behavioral discontinuities within the output codomain. Additionally, static analysis may be used to identify branching conditions and to correlate input values with execution paths within the system. Ideally, we aim to ensure that our test suites ensure complete path coverage through the system using boundary values to detect anomolous behavior at points within the program domain in which design errors are likely to concentrate.

Broadly, two techniques are widely used to identify critical values and edge cases. Static code analysis and domain/codomain analysis. In static code analysis we examine the source code that describes our system and systematically, try to identify branch points, potential overflows (especially at comparisons), and execution paths. We then try to ensure that we test the system at values that trigger branching, that we select values to ensure execution of all execution paths, and that occur near critical domains.

Composition

Each test verifies that a subunit within the system satisfies some constraint. In all but the simplest systems, these verifications have to be combined into an assertion for the system as a whole. This process of verification composition (alternately, proof composition), is an essential process for extrapolating from independent test results to an assurance for the system as a whole. In model checking, barring the rare instances in which exhaustive testing is possible, we rely on a divide and conquer strategy in which we decompose our model into modules. Each module is characterized by an internal behavior and an interface through which it interacts with the other modules that comprise our target system. We use stimulus testing to verify that the module operates correctly over the range of inputs and execution contexts associated with its input/output interface and these then reason over the composition of these subunits using the invariants preserved by their interfaces to conclude that the combined system conforms to its specification.

The model checking methodology can be illustrated using the simple example introduced in the last section. Using the process outlined above, we begin by defining a set of properties that define the functional domain. The specification for the divisibility checker can be defined as follows: forall bs : list Bit, divisible? (decode bs) 3 ¡-¿ output (device bs) = 1. Using the property based definition of the input output domain is sufficient to uniquely define the functional domain uniquely. The next step is to model the system in question. Here we use the Kami representation presented in figure 3. Once we have modeled the system, we must translate our constraints into model constrints. Here we map the input and output referenced in the specification onto specific port values.

Formal Logic

Formally verified software development is the practice of writing mathematical proofs to verify that software is correct. Analogously, mathematical proofs can be used to verify that hardware designs are correct. A number of programming languages support reasoning about software. I specialize in using Coq to model systems, to formalize logical assertions about those systems, to derive mathematical proofs verifying logical propositions, and to verify that those mathematical proofs are logically sound.

A mathematical proofs is a sequence of statements in which every statement is either a logical axiom, or follows from one or more of the preceding statements by applying a logical deduction rule. A formal language consists of a syntax and a semantics. A syntax consists of a set of tokens and a grammar for generating terms. A semantics is a function mapping terms onto values within a denotation set. A formal logic language is a formal language in which we have the fundamental logic values, true and false, a minimum set of logical connectors, such as conjunction, disjunction, implication, etc, and a semantics function that enables us to map expressions onto truth values.

There is an isomorphism between mathematical proofs expressed in Intuitionist Logic and the terms of Typed Lambda Calculus. This isomorphism is referred to as the Curry-Howard isomorphism. This relation equates implication with lambda abstraction, theorem application with function application, propositions with types, and proofs with the values that inhabit a given type. Coq, exploits this relation between terms within Typed Lambda Calculus and proofs within Intuitionist logic to define an language in which we can express algorithms, logical propositions, and mathematical proofs. This language is called Gallina. Coq additionally defines a macro language named Ltac, that can be used to generate Gallina terms, and a command language called Vernacular, that can be used to add definitions, add declarations, change configuration values, etc.

As discussed above, every verification methodology must perform the following operations: specification, modeling, constraint translation, derivation, and composition.

Specification

When using formal logic for specification, we encode the system constraints and requirements into a set of logical propositions. A specification may take the form of a set of logical assertions and propositions, or we may define a canonical model of the system that we are seeking to verify and then prove that the model of our implementation is equivalent to our canonical model.

For example, we can formalize the specification for our example Divisibility Checker, in Gallina, as follows:

```
Theorem fsm_correct :
  forall bs : list Bit,
    decodeBit (output fsm (execute fsm bs)) = 1 <->
    divide 3 (decode bs).
```

Here we Gallina to express a logical predicate from first order predicate calculus to express that the output of our Divisibility checker, **fsm**, must equal **1** when the number represented by our input sequence **decode bs** is divisible by **3**. By using formal logic, we can precisely express the correctness condition for our Divisibility checker precisely.

Modeling

Once we have defined a specification for the system, we must then model the system. Using formal logic, we may choose a variety of representations. While verifying the floating point unit of the Sifive's S-series cores, I modeled the unit as a set of recurrence relations. While verifying components used within Kami Processor, I represented them as Kami expressions. I have modeled our divisibility checker as both a Kami module (see figure 3) and as an abstract finite state machine (see figure 2).

Constraint Translation

After we model the system, we must then translate the specification into model constraints. When I verified the floating point unit, I had to ensure that the values stored within one of the unit's registers were always small enough to fit within the space allocated for the register. When I translated this constraint, I had to apply the isomorphic mapping between the values stored in the implementation's registers to compute the equivalent value expressed in my canonical implementation model.

Similarly, if we were to use formal logic to verify the Kami model of the Divisibility Checker, we would have to translate the output constraint into a constraint on the argument value passed to the call to the **sendOutput** method (see figure 3).

Derivation

Once we have translated our specification constraints into specific model constraints we must derive proofs that these constraints are satisfied by the model.

Mathematical theorem proving involves the construction of mathematical proofs. A mathematical proof can be visualized as a tree structure whose leaves correspond to mathematical axioms and whose nodes correspond to derived assertions.

Each node corresponds to the application of a single logical deduction rule. Every deduction rule is a rewrite rule that transforms one or more statements into a new derived statement. The children of each node correspond to those statements that the deduction rule associated with the node were applied to. There are a small number of logical deduction rules. Generally every logical connective has a set of introduction and elimination rules. In Figure 4, I diagram the principal introduction and elimination rules used in Intuitionist Logic.

Loosely speaking, theorem proving tactics correspond to the application of one or more of these deduction rules. For example, "proof by contradiction" corresponds

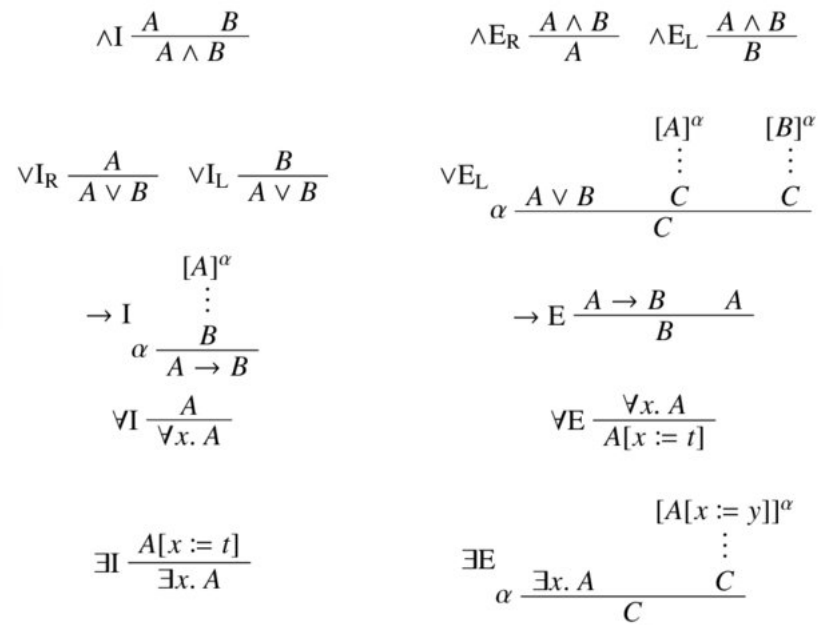


Figure 4: The introduction and elimination rules of Intuitionstic Logic. Reproduced from [1].

to the application of **False_ind** in Gallina. Whereas "case analysis" corresponds to the use of a **match** statement. Accordingly, the process of derivation when verifying hardware designs using formal logic involves the construction of proof terms through the application of theorem proving tactics.

This process is illustrated in Figure 5 in which I present an excerpt of my proof that the abstract finite state machine model of the Divisibility Checker is correct.

A similar example can be seen by examining the sample proof that the floating point unit is correct. Here I use a variety of theorem proving tactics such as mathematical induction, proof by contradiction, and proof by cases while exploiting theorems from real analysis.

At this point it is appropriate to highlight that exhaustive model checking is logically equivalent to the tactic of proof by case analysis. In case analysis, we verify that a property holds for a system by proving that the the property holds for every possible value that can be assumed. Whenever a system's functional domain is small enough to effectively enumerate, we can use case analysis effectively.

As observed earlier however, exhaustive model checking is not feasible for large functional domains, and is logically impossible for infinite domains. In these instances, proof techniques such as mathematical induction enable us to establish results that are strictly impossible to establish using model checking.

```
Theorem fsm_correct :
  forall bs : list Bit,
    decodeBit (output fsm (execute fsm bs)) = 1 <->
    divide 3 (decode bs).
Proof.
  intro bs.
  rewrite <- (mod_divide (decode bs) 3 ltac:(lia)).
  set (Hi := preservesInvariant fsm fsmInvariant fsmInvariantInit
    fsmTransPresInvariant bs).
  unfold fsmInvariant in Hi.
  split.
  + refine (
      FsmS_rect
        (fun s =>
          s = execute fsm bs ->
          decodeBit (output fsm (execute fsm bs)) = 1 ->
          decode bs mod 3 = 0)
        _ _ _ (execute fsm bs) ltac:(reflexivity));
      intro He;
      rewrite <- He in *;
      lazy iota zeta beta delta -[decode modulo];
      rewrite <- Hi;
      intro Htemp.
      - reflexivity.
      - contradict Htemp; lia.
      - contradict Htemp; lia.
  + intro H.
    refine (
      FsmS_rect
        (fun s => s = execute fsm bs ->
          decodeBit (output fsm (execute fsm bs)) = 1)
        _ _ _ (execute fsm bs) ltac:(reflexivity));
      intro He;
      rewrite <- He in *.
      - reflexivity.
      - rewrite H in Hi.
        contradict Hi.
        simpl. lia.
      - rewrite H in Hi.
        contradict Hi.
        simpl. lia.
Qed.
```

Figure 5: An excerpt from my proof that the finite state machine model of the Divisibility Checker is correct. Note that this proof consists of a sequence of Ltac tactics which construct a term in Gallina that represents a mathematical proof.

Composition

Having derived proofs that verify that subunits satisfy some of the constraints imposed by a specification, we must then integrate these results into a composite proof concerning the system as a whole. Here the structure of mathematical proofs inherently supports compositions. Since mathematical proofs posses inherent tree structures and logical deduction rules are inherently compositional, composing subproofs to derive system-level proofs is natural using formal logic. Indeed every proof given in the examples so far involved composing smaller lemmas and assertions to generate broader composite claims.

For example, in Figure 6 I reproduce one of the lemmas that I referenced while proving that the finite state machine model of the Divisibility Checker was correct. This lemma proves that every state transition preserves the following invariant:

```
Definition fsmInvariant (bs : list Bit) (s : FsmS) : Prop :=
  decodeState s = decode bs mod 3.
```

This invariant says that, if we take the number represented by the input received so far, and divide it by 3, the remainder corresponds to the current machine state.

This proof uses case analysis to verify that every transition is correct. In each case I apply a set of equations concerning modular arithmetic and then combine the subproofs about individual state transitions into a composite proof about the model as a whole.

Here, it's worth observing that model checking implicitly relies on formal logic to compose verifications of submodules into proofs that a system composed of these submodules posses the desired properties. Having talked with Prakash, it appears that this process of compositional reasoning is currently informal at Apple, but this may be a promising domain for integrating tools such as Coq, APL, Lean, etc. Model checking can be used to establish properties for submodules, and then these properties can be asserted as axioms in a theorem prover which can be used to formally prove composite results.

```
Lemma fsmTransPresInvariant :
  transitionPreservesInvariant fsm fsmInvariant.
Proof.
  unfold transitionPreservesInvariant.
  unfold fsmInvariant.
  unfold decode at 2.
  intros xs s H x.
  rewrite (rev_app x xs).
  rewrite (rev_involutive (x :: rev xs)) at 1.
  unfold decodeAux.
  fold (decodeAux (rev xs)).
  rewrite <- (decodeEq xs) at 1.
  destruct s, x;
    unfold decodeState in H;
    lazy iota beta zeta delta -[decode modulo mul plus].
  (* state 0 *)
  + refine (ex_ind (fun k => _) (mod_ex 0 (decode xs) 3 ltac:(lia) H)).
    intro Hk.
    rewrite (add_0_r (k * 3)) in Hk.
    rewrite (add_0_r (2 * decode xs)).
    rewrite <- Hk.
    rewrite (mul_assoc 2 k 3).
    rewrite (mod_mul (2 * k) 3 ltac:(lia)).
    reflexivity.
  + refine (ex_ind (fun k => _) (mod_ex 0 (decode xs) 3 ltac:(lia) H)).
    intro Hk.
    rewrite (add_0_r (k * 3)) in Hk.
    rewrite <- Hk.
    rewrite (add_comm (2 * (k * 3)) 1).
    rewrite (mul_assoc 2 k 3).
    rewrite (mod_add 1 (2 * k) 3 ltac:(lia)).
    rewrite (mod_small 1 3 ltac:(lia)).
    reflexivity.
  (* state 1 *)
  ...
  (* state 2 *)
  ...
Qed.
```

Figure 6: An excerpt from my proof that the finite state machine model of the Divisibility Checker is correct. This excerpt proves that the state machine transitions preserve a key invariant. Note that this proof uses case analysis to verify that every transition is correct.

Case Studies

The Kami Processor

In 2018, SiFive tasked the Formal Verification team at SiFive to develop a canonical model of the RISC-V spec for submission to the RISC-V foundation. At the time, the RISC-V foundation believed that the lack of a canonical model for the RISC-V standard was contributing to divergence between several RISC-V implementations. They believed that, if there was a single open source reference implementation then this implementation would provide a valuable codification of the written spec and become the de-facto definition of the RISC-V architecture. Accordingly, the foundation commisioned the Formal Methods Working group to review existing models and to select one. SiFive, tasked my team with developing an open source RISC-V model using Kami, and I was assigned the role as the lead developer for the project.

Accordingly, I worked to develop an open source reference model of the RISC-V specification that encompassed as large a proportion of the RISC-V feature set as possible. It's important to stress that the Kami Processor project itself was an effort to define a reference implementation to be used for traditional model checking. This verification methodology was implicit in the verification praxis encoded into Kami development.

Kami consists of a DSL embedded within Coq and includes a large body of mathematical theorems and logical relations. The critical logical theorem is its Simulation theorem which formalizes the notion of one Kami model being functionally equivalent to another. While the equivalence of two Kami modules can be proved using the full range of theorem proving tactics, such as mathematical induction, in practice, the Formal Verification team supported SiFive by developing models in Kami, using formal logic to establish that those models possessed certain functional attributes, and by using model checking to verify that designs coded in Chisel were equivalent to those models.

Over the subsequent year, I worked with Murali Vijayaraghavan to develop a model of the RISC-V specification in Kami. While I developed this model, Vijayaraghavan, worked in parallel to develop a commercial variant that SiFive intended to offer under the Apex brand name. Our efforts were complementary, in that SiFive intended to use formal logic to prove that the commercial variant developed by Vijayaraghavan was functionally equivalent to the open source model that I produced, which became known as Kami Processor.

The Kami Processor supported a large fraction of the RISC-V ISA. This included memory virtualization, privilege hierarchies. To support this complexity, I introduced an architectural design in which the Kami elements were generated by a set of Gallina "generator functions." These functions parsed database tables that described the semantics of processor instructions, memory operations, compressed instruction mappings, etc. These functions then generated Kami expressions and actions that were composed to form the processor model. This design required me to verify these generator functions to ensure correctness. This effort was accomplished using a combination of verification strategies.

In several instances, I relied on traditional unit testing where I defined a set

of input/output combinations for certain functions and then used Gallina's proof checker to verify that the functions being tested generated the correct output over the test sets. An example of this form of low-level functional unit testing can be seen in https://github.com/llee454/StdLibKami/blob/issue-130/ReplacementPolicy/PseudoLru.v. In this example, I define a set of Gallina functions that generate a cache manager that uses a PseudoLru algorithm to select entries for replacement. The file ends with a set of test cases and Coq's Goal command is used to verify that the generator functions generate the correct output during compilation.

Another example of low-level unit testing for generator functions is given in https://github.com/llee454/ProcKami/blob/master/TlUh.v. In this file, I again generated a series of input/output test cases to verify a set of functions used to generate components of the Kami Processor. In this instance, I tested the mask functions used to generate bit masks used within the processor's Tile Link interface.

In addition to these low-level stimulus tests, I tested the processor model as a whole by running the model over the RISC-V test suite - a collection of assembly programs designed to verify that a RISC-V processor correctly conforms to the instruction semantics prescribed by the RISC-V ISA spec. I utilized this test suite extensively to detect bugs within the model as a whole and developed a number of Bash scripts for automating the running of these tests and the analysis of these test results.

A fuller exposition of the Kami Processor and its current status can be found in [2].

Utila

In addition to traditional stimulus based unit testing, I used formal derived formal proofs verifying that various generators were correct. I collected many of these proofs in the Utila module https://github.com/llee454/Kami/blob/issue-130/Utila.v.

The Utila module defined a collection of common combinators that were used extensively within the generator functions used to generate the Kami Processor model. Each of these functions generated a Kami expression or action. These expressions represented common circuit design elements that were so ubiquitous that proving their correctness was deemed worth the additional verification effort needed to derive mathematical proofs.

For example, the Utila module defines the following combinator:

```
Definition utila_expr_find_pkt
  (k : Kind)
  (pkt_exprs : list (Maybe k ## ty))
  :  Maybe k ## ty
```

utila_expr_find_pkt accepts a list of circuit components that each generate a Kami Maybe value and outputs the value that is marked as valid. This

component is used extensively in Kami Processor's Decoder and Decompressor where it's referenced through a generator function named **inst_db_get_pkt**. The Utila library derives an algebraic proof that this component is correct. The proof is named **utila_expr_find_pkt_correct** and it uses formal logic to verify that the generated component satisfies its functional specification:

```
Theorem utila_expr_find_pkt_correct :
  forall (k : Kind)
         (xs : list (Maybe k ## type))
         (x : Maybe k ## type),
    (unique (fun y => In y xs /\ {{#[[y]] @% "valid"}} = true) x) ->
    [[utila_expr_find_pkt xs]] = [[x]].
```

This theorem asserts that, if there is one, and only one, value that is marked as valid, this component will output that value.

S-Series Cores Verification

Beyond this effort, Vijayaraghavan and I were the primary engineers tasked with supporting the Chisel based teams. My verification work supporting these teams focused on higher-level abstractional units. I worked primarily to support the teams that were developing the S-Series cores. As part of this effort, I developed modules that were used as reference implementations for the floating point unit used within these cores.

In 2018, I was asked to verify the floating point unit of SiFive's S-series cores. SiFive, had used model checking to test their floating point unit, but their implementation was widely considered opaque. It had been written in a highly obfuscated style and the engineering team doubted its correctness. The unit that performed floating point division and square root operations were based on John Hauser's open source Berkley SoftFloat library.

I abstracted the implementation onto a set of recurrence relations, then mapped those recurrence relations onto a canonical model for the division and square root operations. I then proved that the canonical model was correct. Afterwards, I translated the physical constraints associated with the device into value constraints for my model and then verified that the model satisfied these value constraints.

I worked with Vijayaraghavan to develop a model of the subunits that performed floating point multiplication, division, and square root operations. Additionally, I contributed to the pipeline analysis for the S-series cores as these cores stalled on long iterative floating point operations.

An exerpt of my work verifying the floating point unit is available at https://github.com/llee454/FPU-Verification. In this work, I collaborated with Vijayaraghavan. He drafted the Kami model of the division and square root unit to verify the Chisel implementation using traditional model checking while I concentrated on the mathematical analysis, validated the Kami model, and persued a second line of attack to verify the Chisel implementation.

Aside from proving that the algorithm executed by our Kami model was correct - i.e. that the value it computed converged to the correct value within the correct number of cycles, I proved that the register allocations within both our model and Hauser's implementation were correct.

Kami Framework Verification

It's important to emphasize that a substantial fraction of the verification effort associated with the Kami processor involved proving results about the Kami semantics and the generator functions that generated the Kami expressions and actions that comprised the Kami model.

In addition to verify hardware, I worked with the rest of the Formal Methods team to revise the Kami language and to prove various results about its semantics and functional correctness. One example of this work can be found in https://github.com/llee454/Kami/blob/issue-130/StdLib/Fin.v where I worked with Anthony Machado to reformulate the Fin and Vector libraries defined by the Coq Standard library. I proved all of the results established by the Standard library for our new formulation and then proceeded to update every proof in the Kami system that relied on the original formulation for finite numbers and vectors. In this work, Machado assisted by rewriting the Vector files.

Additionally, I contributed to the language syntax, notations, and contributed additional results in Kami/Lib. All of these contributions involved using formal logic to express and then prove assertions about Gallina functions. This work ultimately supported our team's hardware verification efforts by proving the underlying results and data structures that we used to model and reason about hardware designs.

Conclusion

My experience using formal logic, unit testing, and developing models that were used to support traditional model checking has convinced me that these methodologies are fundamentally complementary. All formal verification methodologies must encompass the same fundamental set of activities. And each of these methodologies aim to achieve the same fundamental ends. In a certain conceptual sense, traditional model checking is equivalent to case analysis within mathematical theorem proving. This equivalence, holds whenever exhaustive model checking is possible. When the functional domain prevents us from applying exhaustive model checking, formal logic provides additional tools, such as mathematical induction, that may allow us to establish correctness results. However, in practice, deriving mathematical proofs is a fundamentally hard problem. Given the high level of effort that may be required to mathematically prove certain results, model checking over a subset of the functional domain provides a means o establish high levels of correctness assurance within practically acceptable levels of effort.

While working at SiFive, I advocated for the use of blended verification strategies. In this approach, I advocated for using formal logic to prove the correctness

of critical components within larger more complex systems. An example of this approach is illustrated by my work supporting the S-Series cores. Here, traditional model checking was deemed sufficient for verifying most of the Chisel-based design. However, the floating point unit was considered both critical and dubious. Accordingly, it was a prime example of the value of selective verification.

Another approach is to use model checking to establish results for the subunits that comprise a system and then to reason about their combination using formal logic. My work with the Kami Processor illustrates this approach. Here, I used a combination of static code analysis, formal logic based verification, and unit testing to verify subcomponents used within the Kami Processor, and then used formal logic to reason about their interaction. I believe that a similar approach could be adopted at Apple.

References

[1] Giovanni Birolo, *Interactive Realizability, Monads and Witness Extraction*, arXiv, 2013.

[2] Larry Lee, *Kami Processor: Status and Comparison*, Research Gate, 2020.